

프로젝트 마스터 기획서

항목	내용
문서명	2026 금융 AI Challenge — 프로젝트 마스터 기획서
서비스명	Money is Always Right (가칭: AI 기반 청년층 맞춤형 자산 형성 및 금융상품 추천 플랫폼)
문서 목적	공모전 제출 · 4인 팀 협업 기준선 · MVP 범위 합의
대상	심사위원 · 팀원(Frontend 2 / Backend-Data 1 / AI-Simulation 1)
버전	v1.0
관련 산출물	README.md · API.md · 기능 명세서.md · bedrock-kb-etf.md

한 줄 정의

청년의 재무 상태 → 목표 Gap → 실행 가능한 상품·시나리오·대화형 코칭을 한 웹 MVP로 연결하는 AI 자산형성 플랫폼입니다.

1. 프로젝트 개요 (Project Overview)

1.1 추진 배경 및 기획 의도

청년층은 주거비·생활비 비중이 높아 저축 여력이 낮고, 예·적금·ETF·정책금융 등 상품 선택지가 늘어날수록 비교·의사결정 비용이 급증합니다. 기존 금융 앱·핀테크 서비스는 대체로 다음 한계를 보입니다.

기존 서비스 유형	한계
은행·증권 앱	자사 상품 중심 나열, 개인 목표와의 연결이 약함
단순 가계부	소비 기록에 그치고 목표 달성 경로를 제시하지 않음
로보어드바이저	투자 배분에 편중, 예·적금·저축 여력 설계가 분리됨
범용 AI 챗봇	사용자 재무 맥락·공시 데이터가 없어 일반론에 그침

본 프로젝트는 위 공백을 「분석 → Gap → 시뮬레이션 → 대화형 코칭」 한 사이클로 메우는 것을 기획 의도로 둡니다.

청년에게 필요한 것은 “더 많은 상품 목록”이 아니라,

지금 여력으로 목표까지 얼마나 모자라고(Gap), 무엇을 바꾸면 달하는지(시나리오), 어떤 공시 상품·ETF가 성향에 맞는지(큐레이션·코치)입니다.

1.2 서비스 목표 및 기대 효과

구분	내용
제품 목표	B2C 웹 MVP로 온보딩 → 대시보드 → 시뮬레이션 → AI 코치 전 구간을 시연 가능하게 완성
사용자 목표	목표 자산·기간 대비 달성률·월 저축 여력·실행 로드맵을 이해하고, 조건 변경 시 미래 궤적을 비교

기술 목표	React · FastAPI · Firebase · AWS Bedrock을 연결하고, 외부 API 키 부재 시에도 mock/fallback 으로 데모 가능
공모전 목표	오프라인 PT에서 페르소나 프리셋 1~2분 시연으로 차별점(디지털 트윈·Gap·AI 코치)을 명확히 전달

기대 효과

- **의사결정 시간 단축**: 성향·목표·여력에 맞는 예·적금·(해당 시) ETF 참고 목록을 한 화면에서 확인
- **행동 유도**: “저축률을 올리면” 등 What-if로 Gap 축소 경로를 체감
- **신뢰 가능한 AI**: 숫자는 DB·도구 조회, 설명은 정책·용어 RAG — **환각으로 수익률을 만들지 않음**
- **확장성**: 마이데이터 Mock JSON을 실연동 어댑터로 교체할 수 있는 구조

2. 서비스 차별화 전략 (Key Differentiators)

2.1 디지털 트윈(Digital Twin) 기반 시나리오 시뮬레이션

사용자의 소득·지출·저축률·기대수익률·목표를 **가상 재무 쌍생아(디지털 트윈)**로 모델링합니다. UI에서 변수를 조절하면 **기준 계획(baseline) vs 시나리오**의 자산 궤적(1·3·5년 등 시계열)을 즉시 비교합니다.

차별 포인트	설명
가정 명시	“만약 저축률 +10%p”처럼 변경분이 결과에 반영됨을 보여 줌
목표 히트 라벨	목표 자산 도달 시점 차이를 문장으로 요약
코치 연계	AI 코치가 동일 시뮬레이션 도구를 호출해 대화로 What-if 설명

단순 계산기나 정적 그래프가 아니라, **온보딩 프로필과 동일한 가정으로 트윈을 구동하는 것이 핵심입니다.**

2.2 마이데이터(MyData) 표준 규격 Mocking을 통한 개인화 소비 분석

실제 마이데이터 인가·연동 없이도, **표준에 가까운 JSON 스키마**로 가상 거래 원장을 생성·분류합니다. 대시보드는 이 파이프라인 결과로 **고정비/변동비·카테고리 소비**를 시각화하고, 월 저축 여력 추정에 반영합니다.

항목	MVP 접근
데이터 소스	페르소나 기반 가상 거래 생성 (실연동 전 단계)
분류	규칙 기반 카테고리·고정/변동 구분
향후	오픈뱅킹/마이데이터 실 API 어댑터로 교체 (스키마 유지)

2.3 목표 달성 Gap 분석 기반 상품 큐레이션

상품을 “금리 높은 순 나열”만 하지 않습니다. **목표 자산 - 현재(추정) 자산 = Gap, 월 저축 여력 × 기간**으로 달성 가능성을 먼저 계산한 뒤, 성향에 맞는 **예·적금·연금**과 (안정형 제외) **ETF 참고 목록**을 큐레이션합니다.

레이어	역할
Gap / 로드맵	“무엇을 먼저 할지” (저축·상품·부채·지출)
FSS 공시 상품	예·적금·연금저축 — 금감원 Open API (키 없으면 mock)

ETF (확장)	유니버스 내 6개월 변동성 상대 사분위 × 투자성향 매핑 · 투자 권유 아님 면책
----------	---

안정형은 원금 손실 허용도가 낮아 ETF를 추천하지 않고 예·적금·연금 중심으로 안내합니다.

3. 핵심 기능 명세 (Core Features — 5 Phases)

Phase 1. 사용자 온보딩 및 프로파일링

목적: 이후 모든 계산·추천·코칭의 단일 진실 공급원(프로필) 을 확정합니다.

구분	항목 예시
필수	표시 이름, 연령, 직업, 월 소득, 고정 지출, 추정 월 저축, 투자성향, 목표 자산, 목표 기간, 목표 설명
선택/후속	실제 자산·부채 잔액(대시보드에서 조정 가능), 상세 부채 목록

투자성향 (금융투자협회 표준 5분류)

키	한글
stable	안정형
stable_seeking	안정추구형
neutral	위험중립형
aggressive	적극투자형
very_aggressive	공격투자형

UX 흐름

- 회원가입 · 로그인 (Firebase Auth)
- /onboarding 에서 필수 항목 입력 · 유효성 검증
- Firestore `users/{uid}` 에 저장 · `onboardingCompleted`
- 홈/대시보드로 진입 — 미완료 시 온보딩으로 유도

Phase 2. 데이터 파이프라인 구축

목적: 개인화 분석에 필요한 거래·상품 데이터를 안정적으로 공급합니다.

파이프	내용
거래 파이프라인	사용자·월 단위 가상 거래 생성 → 분류 → 합계 (마이데이터 Mock)
FSS 상품	금융감독원 「금융상품한눈에」 예금·적금·연금저축 Open API
ETF 배치 (확장)	KRX 일별매매 배치 동기화 → 변동성·버킷 원장 (요청 경로에서는 KRX 미호출)

UX 흐름

- /transactions 에서 파이프라인 조회·재생성

- 2. /products 에서 상품 유형별 공시 목록 확인 (source: fss | mock)
- 3. (운영) ETF sync 작업으로 지표·KB 문서 스냅샷 갱신

Phase 3. 개인화 대시보드 및 Gap 분석

목적: "지금 어디쯤인가"를 한 화면으로 이해하게 합니다.

구성 블록	설명
포트폴리오	성향별 비중 가정에 따른 자산 구성 시각화
소비	카테고리·고정/변동비 바 차트
Gap	현재(추정) 자산, 목표, 달성률, 예상 도달 개월·일자, on-track 여부
로드맵	저축·상품·부채·지출 우선순위
상품 큐레이션	성향 맞춤 예·적금·연금
ETF 카드	성향별 버킷 매핑 · 3줄 위험 카피(변동성·유니버스 상대 분위·저/중/고)

UX 흐름

- 1. /dashboard 진입 → 프로필(+선택적 자산·부채 오버라이드)로 POST /api/dashboard/compute
- 2. Gap·로드맵·상품·ETF를 스크롤하며 확인
- 3. ETF 카드 클릭 시 상세·시계열 모달

Phase 4. 디지털 트윈 시뮬레이터

목적: 행동 변경이 Gap에 미치는 영향을 꺾적으로 체감합니다.

입력 변수 (예시)	출력
저축률 · 저축률 증감	월 적립액 변화
연 기대수익률	복리 꺾적
소득·지출 증감	여력 변화
시뮬레이션 기간	1·3·5년 등 시계열 포인트

UX 흐름

- 1. /simulation 에서 기준 계획 로드 (온보딩 기반)
- 2. 슬라이더/입력으로 시나리오 조정
- 3. baseline vs scenario 차트·인사이트 문구 확인
- 4. 코치에게 "저축률 올리면?" 질문 시 동일 엔진 호출

Phase 5. 경량화 AI 금융 코치

목적: 화면의 숫자를 자연어 상담으로 연결하되, 투자 권유·환각을 억제합니다.

구성	설명
UI	우하단 FAB 플로팅 챗 · 도구 실행 트레이스 표시

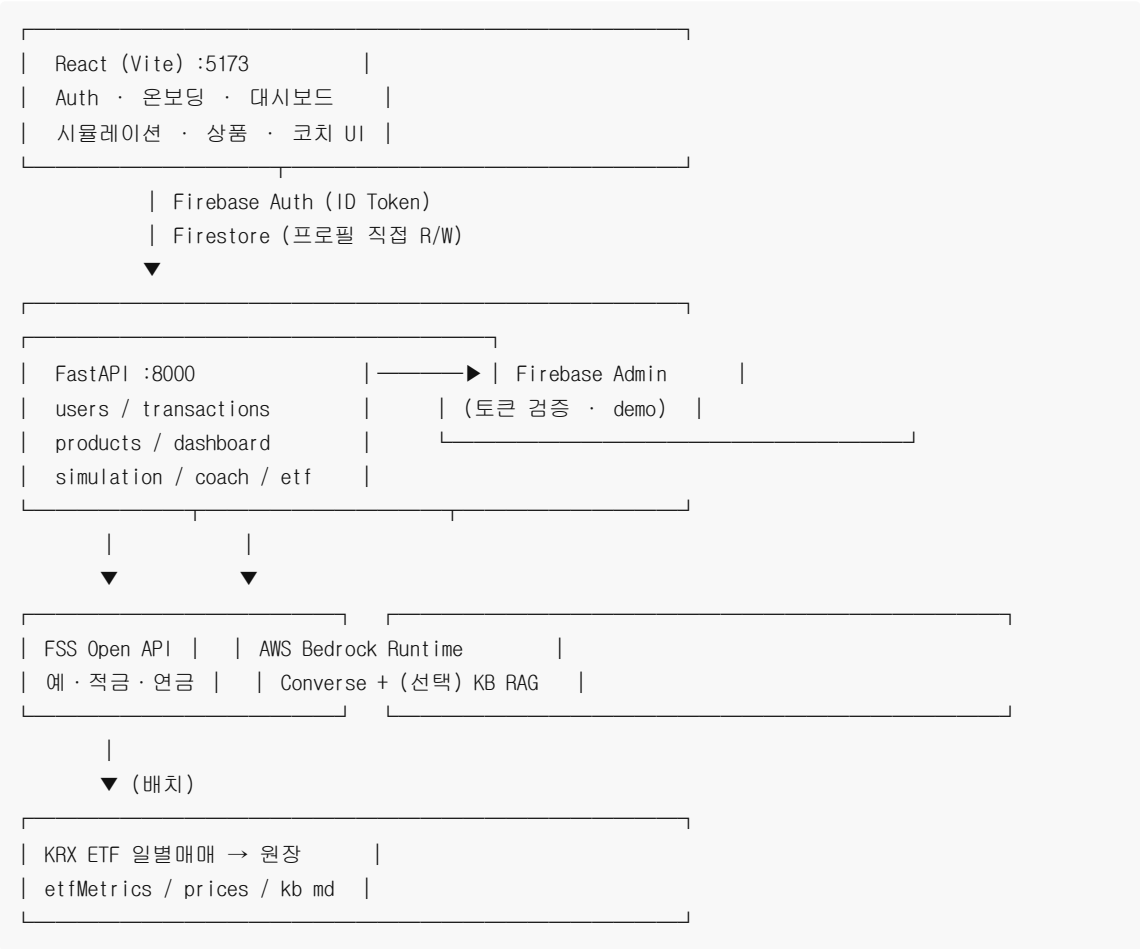
LLM	AWS Bedrock Converse (모델/리전은 환경설정)
도구	금융상태 · 상품검색 · 시뮬레이션 · 로드맵 · ETF 추천/상세 · ETF 지식(RAG)
Fallback	Bedrock 실패 시 키워드 라우팅 + 로컬 도구 결과 안내
면책	투자 권유 아님 · 과거 변동 ≠ 미래 수익 · 자산은 추정값임을 명시

UX 흐름

- 1. 온보딩 완료 사용자만 코치 노출
- 2. 제안 질문 또는 자유 입력
- 3. 에이전트가 도구 호출 → 한국어 답변 (+ thinking 태그 등 내부 스크래치 제거)
- 4. 상단에 `AWS Bedrock 연결됨` / `로컬 안내 모드` 표시

4. 시스템 아키텍처 및 데이터 흐름 (Architecture)

4.1 전체 구성



4.2 Frontend — Backend — Database — AI 통신 흐름

단계	흐름
----	----

1	클라이언트 Firebase 로그인 → ID Token 발급
2	온보딩 프로필 → Firestore users/{uid} (클라이언트 SDK)
3	대시보드·시물·코치·상품 API → Authorization: Bearer <token> → FastAPI
4	FastAPI가 토큰 검증 후, 요청 body의 프로필 스냅샷과 서버 계산/외부 API 결합
5	코치: Bedrock이 toolConfig 로 서비스 계층 호출 → 한국어 최종 응답
6	외부 키 없음: FSS mock · Bedrock fallback · (데모 모드) 인메모리 스토어

환경 분리 원칙: frontend/.env 의 VITE_* 만 브라우저 노출. AWS·FSS·KRX 키는 백엔드 전용.

4.3 마이데이터 Mock JSON 및 금융감독원 API 활용

마이데이터 Mock

- 거래 파이프라인이 **표준에 준하는 필드**(일시, 금액, 가맹점, 카테고리 등)를 생성
- 대시보드 소비·여력 계산의 입력으로만 사용 — **실계좌 잔액·이체를 수행하지 않음**
- CSV/페르소나 프리셋 업로드(로드맵상)로 심사 시연용 데이터 고정 가능

금융감독원 Open API

항목	내용
서비스	금융상품한눈에 — 예금·적금·연금저축
활용	최고금리(또는 공시수익률) 순 큐레이션, 가입 방법·우대조건 요약
장애 대응	FSS_API_KEY 부재/오류 시 source: mock 상품으로 UI 유지

ETF · KRX (확장 데이터)

항목	내용
수집	배치 POST 일별매매 (AUTH_KEY) — 요청 경로에서 day-by-day 호출 금지
지표	약 126거래일(6개월) 연율 변동성 · 유니버스 내 상대 사분위 버킷
AI	숫자는 search_etfs / get_etf_detail · 개념은 Knowledge Base 또는 로컬 kb/etf

5. 프로젝트 운영 및 4인 협업 로드맵 (Operation Strategy)

5.1 4인 분업 구조

역할	인원	주요 책임	주요 디렉터리
Frontend A	1	인증·온보딩·홈·공통 UI	frontend/src/pages, components
Frontend B	1	대시보드·차트·상품·시물 UI · 코치 위젯	DashboardPage, charts, FloatingCoachChat
Backend / Data	1	FastAPI · FSS · 거래 파이프 · ETF 원장 · Firestore Admin	backend/app

AI / Simulation	1	디지털 트윈 엔진 · Bedrock Agent · RAG/KB · 코치 프롬프트	simulation_service, coach_service, kb/
--------------------	---	---	---

도메인 경계를 지키면 PR 충돌이 줄어듭니다. 공유 계약은 API 스키마(docs/API.md)와 프로필 필드입니다.

5.2 Git 브랜치 · 커밋 · PR 프로세스

main → develop → feature/<이슈-또는-기능>

규칙	내용
분기	develop에서 feature/YP2026-XX-... 또는 feature/etf-recommendation 등 생성
커밋	YP2026-XX 요약내용 (예: YP2026-53 ETF 변동성 원장 · 성향 추천 API 및 대시보드 표시)
금지	.env, serviceAccountKey.json 등 시크릿 커밋
PR	feature → develop · 최소 1인 리뷰 · CI(가능 시 lint/test) 통과 후 머지
배포	안정화 시 develop → main

PR 체크리스트 (권장)

- ☐ UI:코치 문구 한국어
- ☐ API 변경 시 docs/API.md 반영
- ☐ 키 없이도 mock/fallback 동작 확인
- ☐ 투자 권유 단정·면책 문구 유지

5.3 공모전 오프라인 PT · MVP 시연 전략

심사 시간은 짧으므로 페르소나 프리셋으로 “가입부터 설명”을 건너뛰고 차별점만 보여 줍니다.

시연 블록	시간 가이드	보여줄 것
혹	30초	청년 Gap 문제 한 문장 + 화면 첫인상
페르소나 로드	30초	프리셋(예: 사회초년생·월저축 80만·위험중립) 원클릭
대시보드	1분	Gap 달성률 · 로드맵 · 성향 맞춤 상품/ETF
디지털 트윈	1분	저축률 인상 → 궤적·목표 도달 변화
AI 코치	1분	“나한테 맞는 ETF” / “레버리지가 뭐야?” — 도구 트레이스 노출
아키텍처·윤리	30초	Bedrock+툴 · 투자 권유 아님 · Mock→실연동 로드맵

페르소나 프리셋 설계 원칙

- 소득·성향·목표가 서로 다른 2종 이상 (안정형 vs 적극형 대비 효과)
- 거래 Mock이 소비 차트에 바로 보이게 시드
- 네트워크 불안정 대비: FSS/Bedrock 키 없이도 끝까지 클릭 가능한 데모 모드 유지

심사 메시지 (권장 한 줄)

“우리는 상품을 더 많이 보여주는 앱이 아니라, 목표까지 모자란 거리를 계산하고, 바꾸면 어떻게 되는지 보여주
고, AI가 그 숫자로만 설명하는 청년 자산형성 코치입니다.”

부록 A. 기술 스택 요약

영역	기술
Frontend	React (Vite), React Router, Recharts
Backend	Python FastAPI, Uvicorn
Auth / DB	Firebase Auth, Cloud Firestore
금융상품	금융감독원 금융상품한눈에 Open API
ETF	KRX Open API (배치) · 상대 변동성 버킷
AI	AWS Bedrock Converse · (선택) Knowledge Base

부록 B. 문서·이슈 매핑 (요약)

영역	Jira (참고)	상태 개요
로그인·온보딩	YP2026-42~44	MVP 구현
거래·FSS	YP2026-45~48	MVP 구현 · CSV 프리셋 등 일부 잔여
대시보드·Gap	YP2026-50~53	MVP 구현 · ETF 고도화 진행
시뮬레이션	YP2026-55~57	MVP 구현 · 스냅샷 영속화 잔여
AI 코치	YP2026-58~60	MVP 구현 · Q&A/KB 고도화 지속
배포·발표	YP2026-61~63	기획서(본 문서) · 배포·발표자료 잔여

본 문서는 공모전 제출 및 팀 내부 기준선용입니다. 구현 세부는 코드·API 명세가 우선하며, 본 기획과 충돌 시 이
슈/PR로 합의 후 개정합니다.